TORSHIELD: A Transparent In-Kernel Authenticated Tunnel with Fail-Closed TC Steering

Roshan Srikanth
Amrita Vishwa Vidyapeetham
Email: roshansrikanth21@gmail.com

Dr. S. UdhayaKumar
Amrita Vishwa Vidyapeetham
Email: sudhayakumar@ch.amrita.edu

***Abstract*—Application-level encryption protects payloads but leaves metadata observable and requires per-application changes. TORSHIELD is a Linux kernel module that transparently encapsulates peer-destined IPv4/IPv6 traffic in UDP with ChaCha20–Poly1305, a 16-byte authenticated header, per-peer 64-bit sequence numbers, a 1024-packet replay window, and 64-byte padding. The same wire format is shared by Netfilter, a route-based L3 device, and TC clsact/eBPF steering. For TC, a kernel guard at `LOCAL_OUT/FORWARD` drops peer-destined packets when the classifier or qdisc is removed. Captured results show byte-exact 1 MB IPv4 and 256 KB IPv6 transfers, 1054/1054 independent AEAD verification, 8/8 local and 8/8 forwarded packets blocked after classifier deletion with zero outer packets, and 3.40 Gbit/s over a 30 s four-stream native VM run. We also report four defects found and fixed. TORSHIELD is not Tor: it provides no anonymity and does not replace WireGuard or IPsec.**

***Index Terms*—Linux kernel, network security, authenticated encryption, ChaCha20–Poly1305, Netfilter, eBPF, traffic control.**

I. INTRODUCTION AND PROBLEM STATEMENT

Network operators increasingly classify, throttle, and block traffic using addresses, ports, flow statistics, packet sizes, and timing. TLS protects content but leaves that metadata observable, and it requires each application to participate. A kernel-level mechanism can apply a uniform policy to every socket without changing applications, which is useful for legacy software, testbeds, and controlled deployments.

An earlier draft of this work described a single-byte XOR transform. We withdraw those claims: a single-byte XOR is trivially recoverable from known plaintext, is length-preserving, and provides no integrity. The prototype is retained only as a historical baseline. The system evaluated here is a different datapath: an authenticated UDP tunnel with per-peer keys, replay protection, and three egress-steering mechanisms.

TORSHIELD is deliberately narrow. It is *not* Tor and provides no anonymity, onion routing, or censorship circumvention. It does not disguise protocol signatures, shape timing, or hide outer addresses and ports. It is also not a replacement for WireGuard or IPsec: it has no handshake and no forward secrecy, and its control plane is a set of procfs files. The contribution is an engineering and measurement study of transparent in-kernel tunneling and of a fail-closed TC steering path.

The work makes three contributions:

- A kernel datapath and wire format for peer-destined IPv4/IPv6 traffic: ChaCha20–Poly1305, a 32-bit key identifier, 64-bit per-peer sequence numbers, 64-byte padding, a 1024-packet replay window, and per-peer allowed prefixes with authenticated roaming.
- One wire format shared by Netfilter, a route-based L3 device, and TC clsact/eBPF steering, plus a kernel-side guard at `LOCAL_OUT/FORWARD` that drops peer-destined packets when the egress classifier or qdisc is missing.
- An empirical study on two kernel versions and a native VM, with independent AEAD verification, deletion tests, manual rekey, a rate limiter, and a 30 s sustained run. We report four defects found and fixed, and we withdraw the draft's unverified CPU, latency, footprint, stability, and distribution claims.

II. STATE OF THE ART

Anonymity and censorship resistance. Tor provides layered encryption and onion routing through relays [1]. Pluggable transports such as obfs4 and ScrambleSuit modify the observable protocol shape [2], [3], and format-transforming encryption maps ciphertext to a target format [4]. These systems solve a different problem: hiding the fact that a flow is Tor or making it resemble another protocol. TORSHIELD does not attempt either. Work on traffic morphing changes packet sizes and timing [5], and measurement studies show that naive obfuscation can be fingerprinted [6]. Those results motivate our explicit non-goal: padding to 64-byte buckets reduces exact-length leakage but does not make the traffic unobservable.

Network-layer protection. IPsec provides authenticated and encrypted IP traffic with a standards-based policy and key-exchange framework [7]. WireGuard is a compact kernel UDP tunnel with a Noise-based handshake, cryptokey routing, and forward secrecy [8]. OpenVPN is a userspace TLS-based tunnel. TORSHIELD shares the UDP/AEAD idea with WireGuard but differs in deployment and scope: it exposes transparent Netfilter steering and a TC/eBPF steering path, uses manually configured per-peer keys, and has no handshake. We do not claim better security or performance than WireGuard; for production privacy, WireGuard or IPsec is the appropriate choice.

Programmable packet processing. eBPF and TC allow packet classification and redirection in the kernel [9]. TOR-

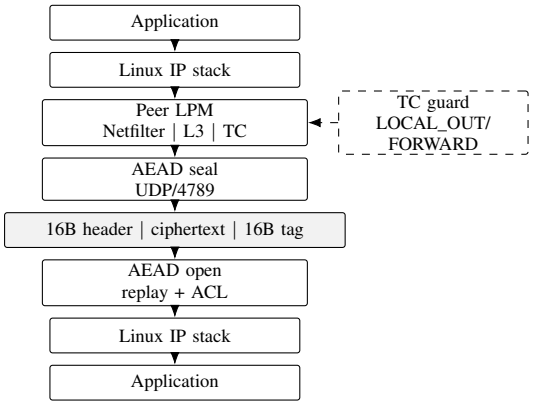


Fig. 1. TorShield v4 datapath. All egress modes share one AEAD wire format; the TC guard drops peer-destined packets when the classifier or qdisc is missing.

SHIELD uses a clsact egress classifier only for steering; the AEAD remains in the device's `ndo_start_xmit` because the BPF helper set cannot call the synchronous in-kernel ChaCha20/Poly1305 path. The fail-closed guard is a separate Netfilter hook, which creates the ordering problem analyzed in Section III-B.

III. Proposed Approach

A. Overview and Wire Format

TORSHIELD is a loadable kernel module. It is inert until a key and at least one peer are configured. Fig. 1 shows the shared datapath. The default mode is `tunnel`. `hybrid` and `tc` add alternative egress-steering mechanisms but keep the same wire format and the same Netfilter ingress decapsulation. `stream` is the legacy length-preserving transform and is excluded from security claims.

An outer packet is `outer IP | UDP:4789 | 16-byte header | padded ciphertext | 16-byte tag`. The 16-byte header is the AEAD associated data: a two-byte `TS` magic, a one-byte version, a one-byte flags field, a 32-bit key identifier, and a 64-bit big-endian per-peer sequence number. The nonce is the 4-byte XOR fold of the outer source and destination addresses followed by the 8-byte sequence number. The plaintext is the complete inner IP packet, zero-padded to a multiple of 64 bytes; the inner header carries the true length and is parsed only after authentication. ChaCha20–Poly1305 follows RFC 8439 [10]. The outer IPv4 identification field is assigned from a per-packet counter so that outer fragments from different datagrams remain distinguishable.

Each peer entry contains a 32-byte key, a deterministic 32-bit key identifier (full FNV-1a of the key, with zero reserved), an outer endpoint, and up to eight allowed inner prefixes. Egress selects the peer by longest-prefix match on the inner destination. Ingress looks up the peer by the wire key identifier, authenticates, and then checks the decrypted inner source against that peer's allowed prefixes. The key identifier is a selector, not an authenticator: an observer can read it, but cannot produce a valid tag without the key. After authentication, a peer with a per-peer key may roam to a new outer source address. A TCP SYN is clamped to an MSS of 1360 bytes for IPv4 and 1340 bytes for IPv6 so that the outer packet fits a 1500-byte MTU.

B. Steering and the Fail-Closed Guard

In `tunnel` mode, Netfilter `POST_ROUTING` matches the inner destination against peer prefixes, encapsulates, and returns `NF_STOLEN`. In `hybrid` mode, a route into a `torshield` L3 netdevice is encapsulated by `ndo_start_xmit`; other peer traffic still uses Netfilter. In `tc` mode, a clsact eBPF classifier on the underlay redirects matching inner packets to the `torshield` device. Outer frames carry mark `0x5454` so they are not redirected twice. If a peer matches but the classifier has no redirect target, the program returns `TC_ACT_SHOT` rather than `TC_ACT_OK`; a matching packet must never fall through to the wire.

The difficult case is classifier removal. Netfilter `POST_ROUTING` runs before TC egress. In TC mode the Netfilter egress hook is bypassed, so relying on the classifier alone would be fail-open if the qdisc or program disappeared. TORSHIELD therefore registers IPv4/IPv6 `NF_INET_LOCAL_OUT` and `NF_INET_FORWARD` hooks at the first priority, both before `POST_ROUTING`. For a peer-destined packet, the guard uses `nf_hook_state.out` and accepts only if the output device is a `torshield` device or still has a live TC egress classifier. Otherwise it returns `NF_DROP` and increments `tc_guard`; a local `sendto()` then fails with `EPERM`. The guard proves only that a classifier is attached, not that its maps are complete. The loader reads every `allowed=` prefix before switching to `tc`, and a peer add, delete, or clear forces the mode back to `tunnel` until the loader is rerun. The guard and the later TC egress hook read the attachment separately, so a classifier deleted after the guard accepts a packet can still let that one in-flight packet leave in clear; the deletion tests prove fail-closed behavior for subsequent packets, not a zero-race guarantee.

C. Replay, Rate Limiting, and Rekey

The receiver keeps a 1024-packet sliding window per peer. The window is advanced only after successful authentication, so an off-path packet cannot consume a window slot. An optional per-source fixed-window limiter runs before `skb_linearize` and Poly1305; it bounds the crypto work a single source can force, but it is not a cookie challenge and the source is spoofable. Per-peer keys can be rotated manually: `rekey-peer` moves the current key to a previous slot and starts sending with the new key; the receiver accepts both key identifiers until `rekey-commit` erases the previous key. A second rotation before commit is rejected. There is no handshake and no forward secrecy.

IV. Preliminary Results

A. Environment and Correctness

Correctness tests ran on Ubuntu 24.04 under QEMU with kernels 6.8.0-147 and 6.17.0-42. Performance tests ran on

TABLE I
REPRESENTATIVE CAPTURED RESULTS. ALL VALUES ARE FROM TEST RUNS, NOT PROJECTIONS.

Check	Captured result
Tunnel TCP	1 MB IPv4 and 256 KB IPv6 byte-exact
AEAD verification	1054/1054 tunnel; 807/807 hybrid v4; 209/209 hybrid v6; 210/210 TC v4; 68/68 TC v6
Wrong-key control	0/210 TC v4; 0/68 TC v6
`LOCAL_OUT` delete	0/8 sent, server 0, `tc_guard` +8, `dev_out` +0
`FORWARD` delete	`n3` outer 0, `tc_guard` +8, `dev_out` +0
30 s native run	3.40 Gbit/s, four streams, `SUnreclaim` 63–68 MB, 0 drops/replays
Defects fixed	GSO skb leak; 64-packet replay window; fragmented-skb read; outer IPv4 ID zero

a native WSL2/Hyper-V VM with kernel 6.18.54.1 and 16 vCPUs; the network is veth, not a physical NIC. Table I summarizes captured results. Independent AEAD verification accepted 1054/1054 tunnel frames, 807/807 hybrid IPv4 frames, 209/209 hybrid IPv6 frames, 210/210 TC IPv4 frames, and 68/68 TC IPv6 frames with the correct key. The TC captures accepted 0/210 and 0/68 with the global PSK, confirming that key selection is in use. The native IPv6 pcap capture was empty in the WSL2 harness, so the IPv6 wire claim rests on the QEMU 68/68 capture; native IPv6 was verified functionally by echo and counters. The implementation also verifies 4 KB and 60 KB inner UDP datagrams, which produce 3 and 41 outer fragments respectively.

B. TC Fail-Closed Behavior

In the `LOCAL_OUT` test, a local socket sends eight packets while the classifier is deleted; all eight fail with `EPERM`, the peer server receives zero, `tc_guard` increases by eight, and both `dev_out` and `tun_out` remain unchanged. Deleting the clsact qdisc produces the same result. Reattaching the classifier restores a 16/16 echo; detaching falls back to the Netfilter path at 8/8.

The `FORWARD` test injects raw frames on `n1`, so the sender's `LOCAL_OUT` hook is bypassed. With no classifier, `n2` receives eight forwarded inner packets, `n3` sees zero outer packets, `tc_guard` increases by eight, and `dev_out` does not move. With the classifier present, `tc_guard` stays flat, `dev_out` increases by eight, and `n3` records eight `tun_rx`. Deleting the classifier returns to zero outer packets and a guard increase of eight.

C. Performance and Defects

On the native VM, the four-stream baseline was 100.98/99.76/97.84 Gbit/s. The tunnel reached 909.7/921.5/965.4 Mbit/s for one stream and 3345.6/3018.1/3203.1 Mbit/s for four streams. A 30 s four-stream run sustained 3.40 Gbit/s with `SUnreclaim` flat at 63–68 MB and zero drops or replays. The RTT average was 0.052 ms over 100 pings. These numbers are not bare-metal measurements.

The evaluation found four defects, all fixed. A GSO path leaked an skb and drove `SUnreclaim` to about 7.4 GB before a panic; releasing each segment after encapsulation fixed it. A 64-packet replay window produced 229 false replay drops and 107 retransmits on four streams; a 1024-packet window produced zero. Reassembled fragmented skbs failed AEAD because the ciphertext was read only from the linear head; `skb_linearize()` before the AEAD fixed it. Outer IPv4 datagrams carried ID zero; a per-packet counter and checksum fix corrected it.

D. Withdrawn Claims

The initial draft reported 1% additional CPU, 0.5 ms per packet, a 50 KB kernel footprint, 5% throughput loss, 72-hour stability, and a matrix of Ubuntu, CentOS, and Debian kernels. Those numbers are not reproduced and are not used. We did not measure CPU utilization or per-packet processing time; the module-size and long-duration stability claims are not part of this study; and the build matrix here is 6.8.0-147, 6.17.0-42, and native 6.18.54.1.

V. SECURITY ARGUMENT

Confidentiality. The inner IP packet is encrypted with ChaCha20–Poly1305 under a per-peer 256-bit key. The header, including magic, version, flags, key identifier, and sequence number, is authenticated as associated data; the inner header is inside the ciphertext. An attacker without the key cannot recover the inner packet or forge a valid tag. The key identifier is a selector and reveals which key is in use, but not the key.

Integrity and source authorization. A frame is accepted only if the tag verifies, the inner version and lengths are consistent, and the decrypted inner source matches one of the peer's allowed prefixes. A valid tag from a different peer therefore cannot inject a source outside its prefix. A peer with a per-peer key may roam to a new outer address after authentication; a global key is accepted only from the configured endpoint.

Replay. The receiver maintains a 1024-packet window per peer and advances it only after authentication. Replayed, reordered, and old frames outside the window are dropped. An off-path attacker cannot consume window slots with invalid tags.

Fail-closed steering. In tunnel mode, authentication and parsing errors drop the packet. In TC mode, the guard drops peer-destined packets when the classifier or qdisc is absent, and the classifier returns `TC_ACT_SHOT` when a peer matches but no redirect target exists. The deletion tests confirm the counter behavior for subsequent packets; the guard-to-egress race is the remaining gap.

Non-goals. The design does not hide outer addresses, ports, message sizes, or timing; does not prevent traffic analysis; and does not provide forward secrecy or post-compromise security. It is not an anonymity system.

VI. DISCUSSION AND LIMITATIONS

A. Deployment Guidance

For current deployments, use `tunnel` mode. It fails closed on authentication and parsing errors and does not depend

on a separately attached classifier. TC mode is experimental: after any peer add, delete, or clear, rerun `torshield-tc attach` before returning to `tc`, because the kernel forces the mode back to `tunnel`. TC mode should not be used where a clear fallback is unacceptable until the guard-to-egress race is closed.

B. Security and Operational Limits

Outer addresses, ports, the `TS` magic, key identifier, sequence number, ciphertext length, direction, timing, and approximate size bucket remain visible. TORSHIELD does not resist traffic analysis or DPI. There is no handshake, no forward secrecy, and no automatic key agreement; a compromised key exposes the traffic it protects, and a compromised endpoint exposes plaintext. The peer table, key, and mode are global across network namespaces. The rate limiter is a fixed-window filter, not a cookie challenge, and the source is spoofable. The guard-to-egress race remains open, TCX is compile-tested only, and TC and hybrid mode are prototypes rather than production-hardened transports. These limits are acceptable for a research artifact but not for a production VPN.

C. Threats to Validity

Throughput is measured on WSL2/Hyper-V with veth, not on bare metal or a physical NIC; checksum and offload behavior on a real NIC is not validated. The TC deletion tests are IPv4 only; the IPv6 guard hooks build but are not separately tested for removal. TCX is compile-tested only. Rekey is IPv4 only. The native IPv6 pcap capture was empty, so the IPv6 wire evidence is the QEMU capture. The performance suite does not isolate CPU time per packet, and the RTT run is short. These limitations bound the claims; they do not affect the byte-exact correctness or the deletion counters.

VII. Doctoral Research Plan

A. Research Questions

The dissertation is organized around three questions:

- **RQ1 (correctness under transparency):** Can a transparent kernel tunnel preserve byte-exact IPv4/IPv6 semantics across GSO, fragmentation, reconnect, and peer change while remaining fail-closed when the TC classifier is removed?
- **RQ2 (cost of in-kernel protection):** What are the throughput and CPU costs of moving AEAD and per-peer policy into the kernel, compared with WireGuard and IPsec under identical traffic and hardware?
- **RQ3 (secure control plane):** Can a handshake with forward secrecy and a per-network-namespace control plane be added without reopening the steering race or weakening transparency?

B. Timeline and Milestones

The first year closes the guard-to-egress race with a last-chance transmit check or a single authoritative steering point, specifies a Noise-style or ephemeral X25519 handshake with a pre-shared key, and moves state to netlink and per-network-namespace storage. It also adds IPv6 and TCX deletion tests. The second year repeats the throughput, CPU, and robustness measurements on bare metal and a physical NIC, compares against WireGuard and IPsec under the same traffic, and performs an external security review. The third year packages the artifact for reproducibility, completes the dissertation, and prepares the defense.

C. Expected Contributions

The expected contributions are (i) a transparent, fail-closed kernel tunnel with measured classifier-loss behavior; (ii) an open-source artifact and test harness for IPv4/IPv6, GSO, fragmentation, replay, and rekey; and (iii) a cost analysis against WireGuard and IPsec that separates data-path from protocol cost. These results address whether transparent kernel tunneling can be deployable and fail-closed at acceptable throughput.

VIII. Conclusion

TORSHIELD is a transparent in-kernel authenticated tunnel with three steering paths and a kernel-side fail-closed guard for TC. The current code is byte-exact on IPv4 and IPv6 TCP, survives independent AEAD verification, drops replay/tamper/unknown-source packets, and blocks subsequent local and forwarded packets when the TC classifier is removed. On a native VM it sustains 3.40 Gbit/s over four streams for 30 s with flat unreclaimable memory. The paper also reports the defects that the evaluation found and the claims that the earlier XOR draft cannot support. Until the handshake and race are addressed, TORSHIELD is a research and teaching artifact, not a production replacement for WireGuard or IPsec.

References

[1] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," in *Proceedings of the 13th USENIX Security Symposium*, 2004, pp. 303–320.

[2] The Tor Project, "obfs4: A pluggable transport," https://gitlab.com/yawning/obfs4, 2024, accessed 2026-10-22.

[3] P. Winter, T. Pulls, and J. Fuss, "Scramblesuit: A polymorphic network protocol to circumvent censorship," in *Proceedings of the 12th ACM Workshop on Privacy in the Electronic Society*, 2013, pp. 213–224.

[4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Protocol misidentification made easy with format-transforming encryption," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security*, 2013, pp. 61–72.

[5] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009.

[6] A. Houmansadr, C. Brubaker, and V. Shmatikov, "The parrot is dead: Observing unobservable network communications," in *Proceedings of the 2013 IEEE Symposium on Security and Privacy*, 2013, pp. 65–79.

[7] S. Kent and K. Seo, "Security architecture for the internet protocol," RFC 4301, IETF, 2005.

[8] J. A. Donenfeld, "Wireguard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.

[9] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, "The express data path: Fast programmable packet processing in the operating system kernel," in *Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2018, pp. 54–66.

[10] Y. Nir and A. Langley, "Chacha20 and poly1305 for ietf protocols," RFC 8439, IETF, 2018.